```
#monalika_kubde - 23070521094
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

from google.colab import files
import pandas as pd

print("Please upload creditcard.csv")

uploaded = files.upload()

filename = next(iter(uploaded))

data = pd.read_csv(filename)

print("\nDataset loaded successfully!")
print("Filename:", filename)
print("Dataset shape:", data.shape)

print("\nFirst 5 rows:")
print(data.head())

print("\nClass distribution:")
print(data["Class"].value_counts())

print("\n========================================")
print("DATASET INFORMATION")
print("========================================")

print("\nFirst 5 rows:")
print(data.head())

print("\nDataset Shape:")
print(data.shape)

print("\nColumn Names:")
print(data.columns)

print("\nMissing Values:")
print(data.isnull().sum().sum())

print("\n========================================")
print("CLASS DISTRIBUTION")
print("========================================")

print(data["Class"].value_counts())

print("\nPercentage Distribution:")
print(
    data["Class"].value_counts(normalize=True) * 100
)

plt.figure(figsize=(7, 5))

sns.countplot(
    x="Class",
```

```
    data=data
)

plt.title("Credit Card Transaction Class Distribution")
plt.xlabel("Class (0 = Normal, 1 = Fraud)")
plt.ylabel("Number of Transactions")

plt.show()

X = data.drop(
    "Class",
    axis=1
)

y = data["Class"]

scaler = StandardScaler()

if "Time" in X.columns:
    X["Time"] = scaler.fit_transform(
        X[["Time"]]
    )

if "Amount" in X.columns:
    X["Amount"] = scaler.fit_transform(
        X[["Amount"]]
    )

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\n========================================")
print("TRAIN-TEST SPLIT")
print("========================================")

print("Training samples:", X_train.shape[0])
print("Testing samples :", X_test.shape[0])

classes = np.unique(y_train)

class_weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weight_dict = dict(
    zip(classes, class_weights)
)

print("\n========================================")
print("CLASS WEIGHTS")
print("========================================")

print(class_weight_dict)

model = Sequential([

    # Input / First Hidden Layer
    Dense(
        64,
        activation="relu",
        input_shape=(X_train.shape[1],)
    ),

    Dropout(0.3),

    # Second Hidden Layer
    Dense(
        32,
        activation="relu"
    ),
```

```
    Dropout(0.3),

    # Third Hidden Layer
    Dense(
        16,
        activation="relu"
    ),

    Dropout(0.2),

    # Output Layer
    Dense(
        1,
        activation="sigmoid"
    )
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=[
        "accuracy",
        tf.keras.metrics.Precision(name="precision"),
        tf.keras.metrics.Recall(name="recall")
    ]
)

print("\n=========================================")
print("DEEP NEURAL NETWORK")
print("=========================================")

model.summary()

early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

print("\n=========================================")
print("TRAINING MODEL")
print("=========================================")

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=256,
    validation_split=0.20,
    class_weight=class_weight_dict,
    callbacks=[early_stopping],
    verbose=1
)

y_probability = model.predict(
    X_test,
    verbose=0
)

y_pred = (
    y_probability >= 0.5
).astype(int).ravel()

accuracy = accuracy_score(
    y_test,
    y_pred
)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
```

```
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\n========================================")
print("MODEL PERFORMANCE")
print("========================================")

print(
    f"Accuracy  : {accuracy:.4f}"
)

print(
    f"Precision : {precision:.4f}"
)

print(
    f"Recall    : {recall:.4f}"
)

print(
    f"F1-Score  : {f1:.4f}"
)

print("\n========================================")
print("CLASSIFICATION REPORT")
print("========================================")

print(
    classification_report(
        y_test,
        y_pred,
        target_names=[
            "Normal",
            "Fraud"
        ],
        zero_division=0
    )
)

cm = confusion_matrix(
    y_test,
    y_pred
)

print("\n========================================")
print("CONFUSION MATRIX")
print("========================================")

print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=[
        "Normal",
        "Fraud"
    ],
    yticklabels=[
        "Normal",
        "Fraud"
    ]
)

plt.title(
    "Confusion Matrix - Credit Card Fraud Detection"
)

plt.xlabel("Predicted Class")
```

```
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.show()

plt.figure(figsize=(9, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy",
    linewidth=2
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy",
    linewidth=2
)

plt.title(
    "Training and Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(9, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss",
    linewidth=2
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss",
    linewidth=2
)

plt.title(
    "Training and Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid(True)

plt.show()

plt.figure(figsize=(9, 5))

plt.plot(
    history.history["precision"],
    label="Training Precision",
    linewidth=2
)

plt.plot(
    history.history["val_precision"],
    label="Training Recall",
    linewidth=2
)

plt.title(
    "Precision and Recall During Training"
)

plt.xlabel("Epoch")
plt.ylabel("Score")
```

```
plt.legend()
plt.grid(True)

plt.show()

print("\n========================================")
print("FINAL RESULT")
print("========================================")

print("Credit Card Fraud Detection completed.")

print(
    f"\nPrecision = {precision:.4f}"
)

print(
    f"Recall    = {recall:.4f}"
)

print(
    f"F1-Score  = {f1:.4f}"
)
```